

“UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

FACULTAD DE INGENIERIA

CARRERA DE INGENIERIA DE SISTEMAS



PROYECTO

**“DESARROLLO DE UN SISTEMA DE CONTROL DE INVENTARIOS
AUTOMATIZADO MEDIANTE VISIÓN ARTIFICIAL”**

NOMBRE:	FABRICIO ALVARO TAMBO ALCON
DOCENTE:	JAVIER ELVIS CANQUI LLUSCO
SEMESTRE:	5to Semestre
MATERIA:	INGENIERIA DE SISTEMAS

EL ALTO – BOLIVIA

CAPÍTULO 1: MARCO INTRODUCTORIO

1.1 Introducción

La cuarta revolución industrial y la adopción de la Inteligencia Artificial (IA) representan el eje estructural para optimizar la logística global y regional. En la ciudad de El Alto, Bolivia, las comercializadoras de suministros industriales y estructuras voluminosas enfrentan desafíos críticos al gestionar sus inventarios. Tradicionalmente, la dependencia de métodos empíricos de conteo manual incrementa el margen de error, reduce la eficiencia operativa y genera discrepancias de datos. El presente proyecto aborda esta problemática mediante la implementación de un sistema web integral potenciado por visión artificial, utilizando el algoritmo YOLO (You Only Look Once), diseñado para digitalizar de forma autónoma la gestión de almacenes de materiales pesados (lonas industriales, rollos de malla y estructuras metálicas).

1.2 Planteamiento del Problema

La comercializadora basa su control de existencias en anotaciones manuales y cuadernos de registro. Debido al elevado peso y volumen de los productos (ej. rollos de media sombra y estructuras de eventos), el conteo físico demanda un esfuerzo extenuante, lo que produce fatiga laboral y conteos omitidos. Esto genera el fenómeno del "stock fantasma": discrepancias críticas entre el registro lógico y las existencias reales en el estante, provocando pérdidas por ventas fallidas o sobreabastecimiento innecesario.

¿Cómo optimizar el control y registro visual de inventarios de productos voluminosos en tiempo real para erradicar el stock fantasma y reducir las pérdidas operativas en la comercializadora mediante visión artificial?

1.3 Objetivos

Objetivo General:

Desarrollar e implementar un sistema web de gestión de inventarios automatizado, integrando el modelo de visión artificial YOLO, para optimizar el control, detección y registro en tiempo real de insumos industriales en la comercializadora.

Objetivos Específicos:

1. Diseñar y construir el frontend estático de la aplicación web (Landing Page y Dashboard) utilizando metodologías ágiles y desplegarlo mediante GitHub Pages.
2. Entrenar un modelo de red neuronal (YOLOv8) mediante *Transfer Learning* para clasificar específicamente rollos de malla, lonas y kits de estructuras.
3. Desarrollar el backend del sistema (API REST) en Python y estructurar la base de datos relacional en PostgreSQL para almacenar el catálogo de productos y registros de detección.
4. Integrar el motor de visión artificial con la base de datos para sincronizar automáticamente el conteo físico detectado con el inventario lógico.

CAPÍTULO 2: MARCO TEÓRICO

2.1 Definiciones Enfocadas al Sistema

Fundamentado en los principios de Ingeniería de Software (Pressman, 2010), el sistema se clasifica como una aplicación web transaccional orientada a datos, enriquecida con componentes de Deep Learning.

- **Visión Artificial y Deep Learning:** Subcampos de la inteligencia artificial que utilizan redes neuronales convolucionales (CNN) para interpretar imágenes.

- **Algoritmo YOLO (You Only Look Once):** Arquitectura de "una sola etapa" (*one-stage detector*) que predice simultáneamente cuadros delimitadores (*bounding boxes*) y probabilidades de clase en una imagen completa, garantizando inferencias en tiempo real con baja latencia.

2.2 Definiciones Enfocadas a la Metodología

- **SCRUM:** Marco de trabajo ágil basado en iteraciones cortas denominadas *Sprints*, priorizando la entrega de valor incremental mediante artefactos como el *Product Backlog*.
- **UML (Lenguaje Unificado de Modelado):** Estándar para visualizar, especificar y documentar los artefactos de un sistema de software, destacando los Casos de Uso para el modelado de comportamiento.
- **Ingeniería de Requisitos:** Proceso sistemático de descubrir, documentar y mantener los requerimientos de un sistema para asegurar que satisfaga las necesidades del negocio.

2.3 Definiciones Enfocadas al Problema (Glosario del Negocio)

- **Lona Industrial / Carpa:** Cubierta de plástico pesado de alta resistencia, gestionada logísticamente como unidad individual o paquete cerrado.
- **Malla de Media Sombra:** Tejido de protección solar gestionado estrictamente por **rollo cerrado** (sin fraccionamiento en sistema).
- **Estructura Metálica (Kit):** Producto compuesto que incluye armazón y cobertura, cuya venta descuenta ambos componentes simultáneamente.
- **Stock Fantasma:** Discrepancia donde un artículo figura como "disponible" en el sistema informático, pero está físicamente ausente en el almacén.

CAPÍTULO 3: MARCO DE DESARROLLO

3.1 Ingeniería de Requisitos

Especificación de Requisitos Funcionales (RF)

(Fórmula: [Sujeto] + [Verbo] + [Acción] + [Condición])

1. **RF01:** El Administrador debe registrar nuevos productos en el catálogo web cuando se adquieran nuevos modelos de carpas.
2. **RF02:** El Encargado de Almacén debe visualizar el conteo automatizado generado por la cámara durante el proceso de auditoría de stock.
3. **RF03:** El Sistema debe actualizar las existencias en PostgreSQL automáticamente tras recibir una detección validada por YOLO.
4. **RF04:** El Sistema debe generar alertas visuales en el Dashboard cuando el stock de un producto alcance el nivel mínimo parametrizado.
5. **RF05:** El Usuario debe iniciar sesión ingresando credenciales válidas para acceder a los módulos administrativos.

Especificación de Requisitos No Funcionales (RNF)

1. **RNF01 (Infraestructura):** El motor de IA debe procesar el flujo de video con una latencia menor a 500ms utilizando la aceleración por hardware de la GPU RTX 3060.
2. **RNF02 (Seguridad):** El sistema debe almacenar las contraseñas aplicando algoritmos de *hashing* seguro (ej. Bcrypt), en cumplimiento con los lineamientos de control de acceso de la norma ISO/IEC 27001:2022.

Priorización: Matriz MoSCoW

- **Must Have (Debe tener):** Login seguro, CRUD de productos, Detección YOLO, Base de datos PostgreSQL.

- **Should Have (Debería tener):** Alertas de stock mínimo en el Dashboard.
- **Could Have (Podría tener):** Generación de reportes de discrepancias en formato PDF.
- **Won't Have (No tendrá por ahora):** Aplicación móvil nativa para clientes externos.

Matriz de Trazabilidad (RTM)

Objetivo Específico	Requisito Funcional	Caso de Uso UML	Entidad del DER
Obj. 3 (BD y Catálogo)	RF01 (Registrar Producto)	CU02: Gestionar Catálogo	Producto, Categoría
Obj. 4 (Integración IA)	RF02 (Visualizar Conteo)	CU03: Escanear Almacén	DeteccionYOLO
Obj. 4 (Integración IA)	RF03 (Actualizar Stock)	CU04: Sincronizar Stock	Producto, DeteccionYOLO
Obj. 3 (BD y Seguridad)	RF05 (Iniciar Sesión)	CU01: Autenticación	Usuario

3.2 Modelado de Comportamiento y Datos

Escenario de Éxito: Flujo Normal de "Sincronizar Stock Visual"

1. El *Encargado de Almacén* accede al módulo "Escaneo YOLO".
2. El sistema activa el flujo de video y el modelo YOLO procesa los *frames*.
3. El sistema identifica y dibuja *bounding boxes* sobre rollos y estructuras.

4. El sistema agrupa el conteo temporal (Ej: 5 Rollos de Malla detectados).
5. El *Encargado* presiona "Validar y Actualizar".
6. El sistema compara el conteo visual con la tabla Producto.
7. El sistema inserta un registro en DeteccionYOLO y actualiza stock_real.
8. El sistema muestra mensaje de éxito: "Inventario Sincronizado".

Diagrama Entidad-Relación (DER) Lógico

El diseño de datos se compone de 4 entidades principales:

- **Usuario (1) --- (N) DeteccionYOLO:** Un usuario valida muchas detecciones.
- **Categoria (1) --- (N) Producto:** Una categoría agrupa múltiples productos (Lonas, Rollos, Estructuras).
- **Producto (1) --- (N) DeteccionYOLO:** Un producto posee múltiples registros de detección histórica a lo largo del tiempo.

Diccionario de Datos (Muestra de Entidades Críticas)

Tabla: Producto

Campo	Tipo de Dato	Restricciones	Descripción
id_producto	Integer	PK, Auto-increment	Identificador único del insumo.
nombre	Varchar(100)	Not Null	Nombre descriptivo (ej. Lona 6x8m).

dimensiones	Varchar(50)	Nullable	Medidas físicas (Largo x Ancho).
stock_teorico	Integer	Default 0	Cantidad registrada manualmente.
stock_real	Integer	Default 0	Cantidad verificada por IA.
id_categoria	Integer	FK	Llave foránea hacia tabla Categoría.

Tabla: DeteccionYOLO

Campo	Tipo de Dato	Restricciones	Descripción
id_deteccion	Integer	PK, Auto-increment	Registro único del evento visual.
fecha_hora	Timestamp	Not Null	Momento exacto del escaneo.
cantidad	Integer	Not Null	Total de objetos detectados.
confianza_ia	Decimal(5,2)	Not Null	Porcentaje de

			precisión (mAP).
--	--	--	------------------

3.3 Arquitectura de Software y Diseño UX

Diagrama de Arquitectura en Capas

El sistema se diseña bajo un patrón de arquitectura N-Capas estructurado así:

1. **Capa de Presentación (Frontend):** Interfaz web desarrollada en React/HTML5 y Tailwind CSS (desplegada en GitHub Pages). Responsable de la UI y la interacción con el usuario.
2. **Capa de Negocio y Lógica de IA (Backend):** Servidor desarrollado en Python (FastAPI/Flask). Contiene la lógica de autenticación (JWT), la API REST para el CRUD, y el motor de procesamiento donde PyTorch carga los pesos del modelo YOLOv8 para la inferencia de imágenes.
3. **Capa de Datos:** Motor de base de datos relacional PostgreSQL. Almacena catálogos, credenciales y registros históricos.

Correspondencia UI vs DER (Wireframes)

En la interfaz del módulo "Agregar Producto", el prototipo visual (Frontend) guarda estricta correspondencia con el esquema de base de datos (Backend):

- El campo input de texto "Nombre del Material" en el diseño UI se mapea directamente e inserta el valor en el atributo `Producto.nombre`.
- El menú desplegable "Tipo de Elemento" extrae sus opciones de `Categoria.nombre_categoria` y guarda el `id_categoria` como llave foránea.
- La vista de la cámara envía el *frame* al backend, y los resultados numéricos devueltos actualizan la visualización del `Producto.stock_real` en el panel lateral del Dashboard.